CS-112
Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Google Classroom Codes

Theory Code

wt3njj4

For queries related to course(Assignments/Projects) just leave an email at

habiba.arshad@uow.edu.pk

during office timings 08:00AM - 04:00 PM

Evaluation Criteria (3+1) Theory

Assessment	%Age
Assignments/ Quizzes	10% + 15% = 25 %
Mid Term Exam	30%
Final Term Exam	45%
Total	100

Assignments and Quizzes

Assignments

Assignments will be due at the beginning of the class. Under normal circumstances, late work will not be accepted.

Students are expected to do their own assignments.

Quizzes

- Oral Quizzes are conducted to check understanding of previous lectures before starting the next lecture.
- Announced quizzes to assess the understanding of taught topics.

Few Things to Remember

- Show sensibility in your conduct.
- Learning should be your primary objective.
- Attendance will be marked within 10 minutes at the start of the class.
- 80% of your attendance is mandatory.
- **Zero tolerance policy** on attendance, discipline of class during lectures.
- Assignments must be submitted on time, no late submissions.
- In case of copied assignment both parties will be given **zero.**
- Don’t miss your Classes, Quizzes, Presentations, Assignments and Projects/Presentations.
- You all need to be a good human being.

Recommended Books

1. Java: The Complete Reference by Herbert Schildt latest edition
2. Java How to program , Eigth Edition, by Deitel and Deital
3. Case Studies and other related material will be provided during the course.

Course Overview

- An Overview of Object Oriented Programming and Java
- Fundamental Programming Structure in Java
- Introducing Classes
- Constructors in Java
- Inheritance
- Polymorphism
- Dynamic Memory Allocation

- Java Packages
- Arrays and Java Collections
- Introduction to GUI
- Layouts and event Handling
- Exception Handling
- Threads in java
- File handling and Database connectivity

Topics To be Covered Today

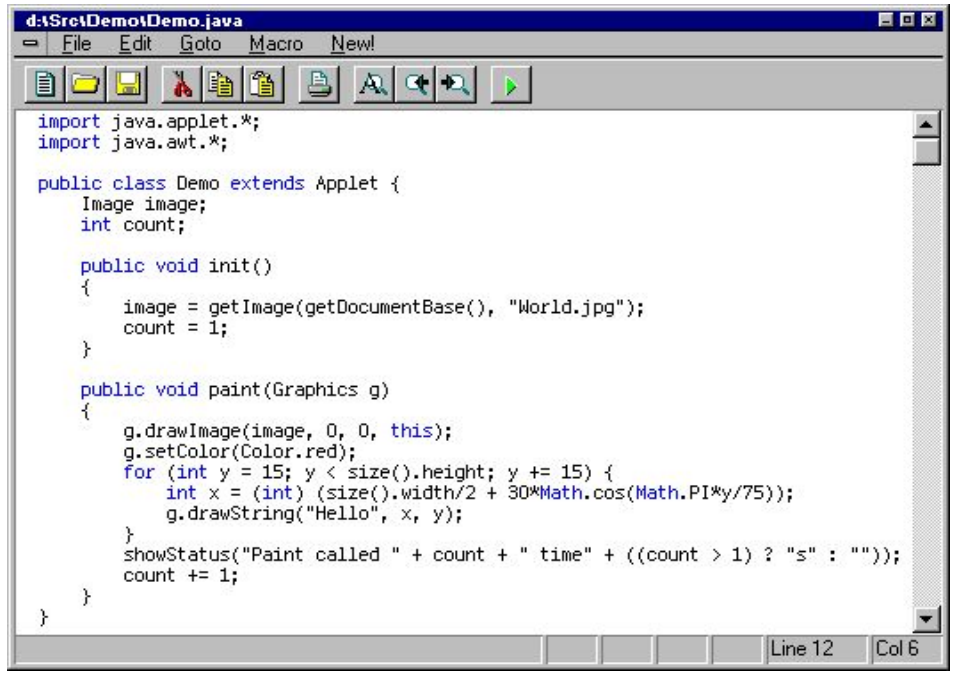
- What is a Program?
- Evolution of Programming Languages
- Why Java?
- Java History
- Features of Java
- Procedural vs object oriented programming language.
- Java Procedure

CLO Covered

CLO1: Understand the underlying concepts of object-oriented paradigm.

What is a Program?

A computer program (also a software program, or just a program) is a sequence of instructions written to perform a specified task for a computer.



Programming Language

The definition of programming language states;

“any kind of language that is used to do computational tasks on the computer and written to produce some output after doing computations is called programming language”.

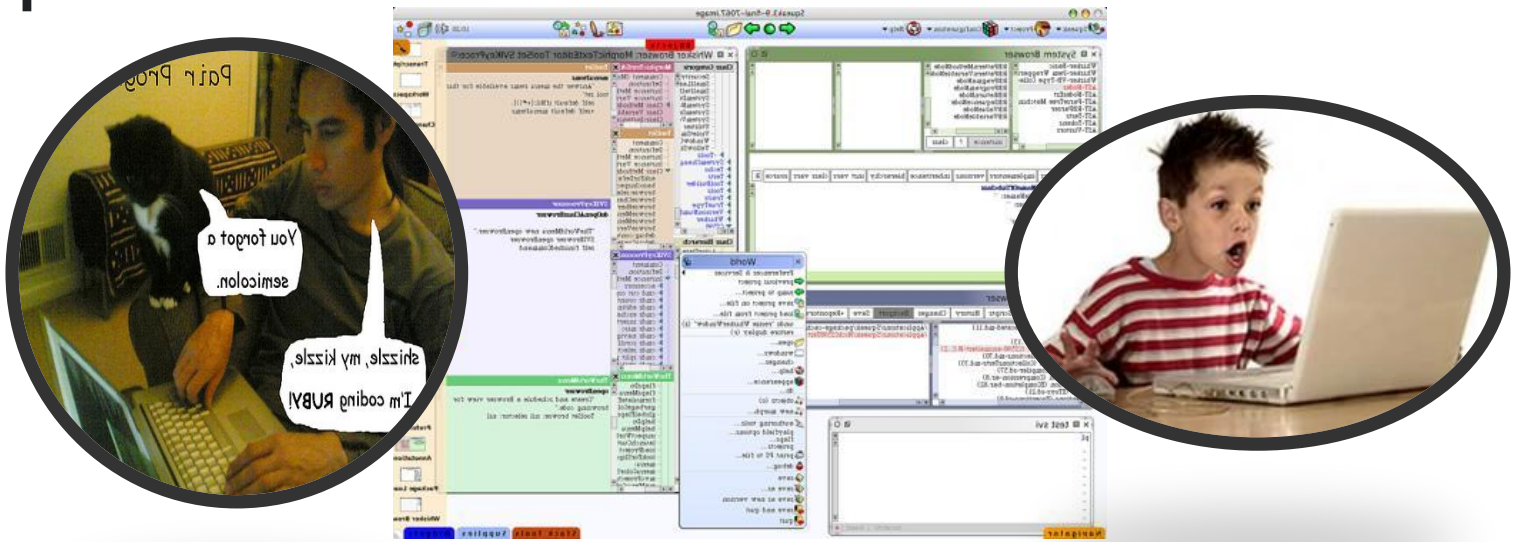
The Evolution of Programming Languages

1. Machine language: 1940’s
2. Assembly language: early 1950’s
3. Higher-level languages: late 1950’s
4. Today: thousands of programming languages

What is Programming?

When we say “programming” we are actually referring to the science of transforming our intentions in a high-level programming language.

It is **the process of creating a set of instructions that tell a computer how to perform a task**.



What are we doing in this course?

There is a proper way or format or structure available to write programming tasks using various languages. This way of writing programs using the proper format is called programming paradigms.

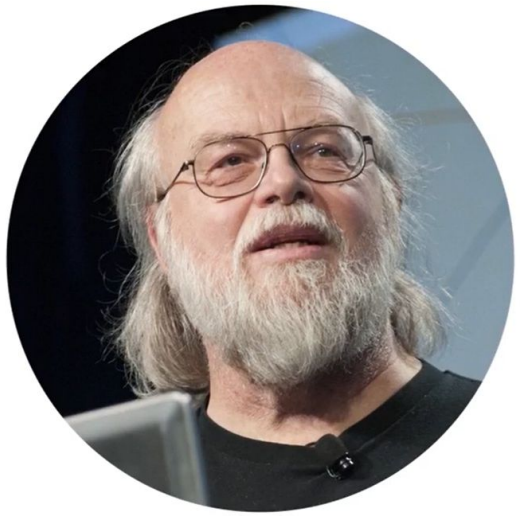
Paradigms is also called programming design philosophy. Various programming paradigms are available through which we can write our programs. Three main programming paradigms are:

- Object-Oriented Programming Model
- Procedural-Oriented Programming Model
- Functional Programming Model

We will study Object-Oriented Programming using '**Java**', a popular high-level object-oriented programming language.

Why Java?

- Java mean **Cup of coffee**
- It's almost entirely object-oriented
- It has a vast library of predefined objects and operations
- It's more platform independent
  - this makes it great for Web programming
- It's more secure
- **It isn't C++**



Java History

James Gosling And Sun Microsystems Invented The Java Programming Language In 1991.

Motto is “ Write Once Run Anywhere”.

Java History

- They first named this language "Oak" because of the oak tree outside Gosling's office.
- Later, the name changed to Green,
- then to Java Coffee, which was named after the coffee from Indonesia,
- and eventually shortened to Java in 1995.

What is Java-Based Upon?

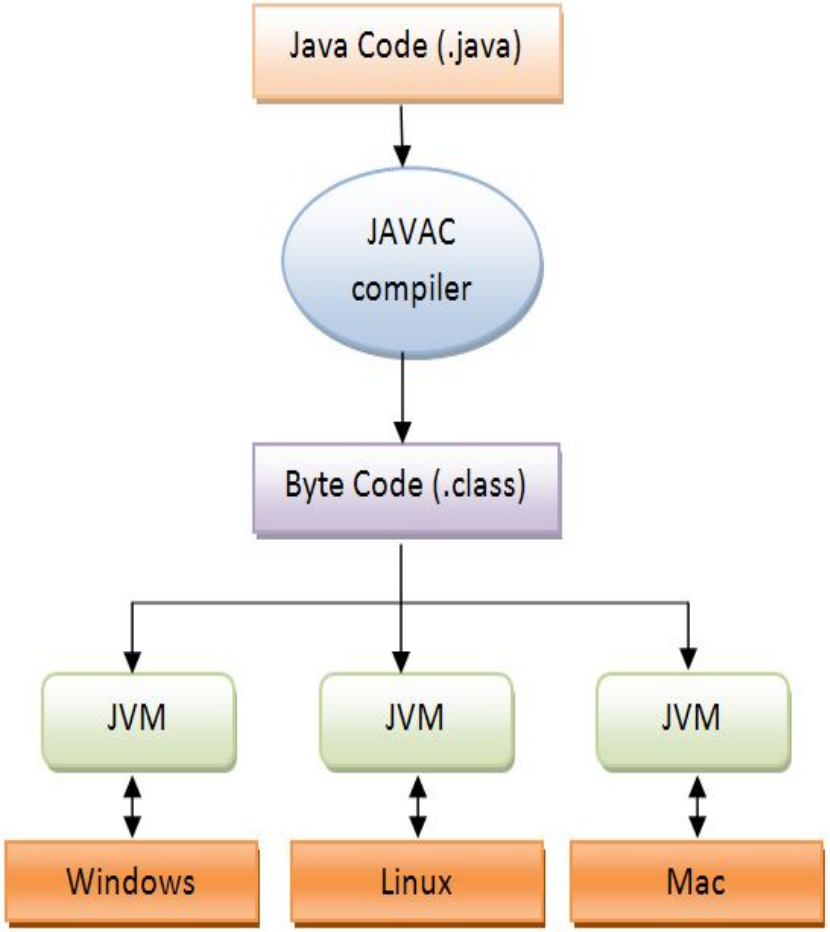
Java is based on C and C++.

The first Java compiler was developed by Sun Microsystems and was written in C using some libraries from C++.

Java files are converted to bit code format using a compiler that the Java interpreter then executes.

Java code runs on Java Virtual Machine (JVM)—the runtime environment.

Steps for creating and running a Java program



Applications of Java technology

Widely used in web consoles, GUIs, web and mobile applications, game development, embedded systems, and desktop applications.

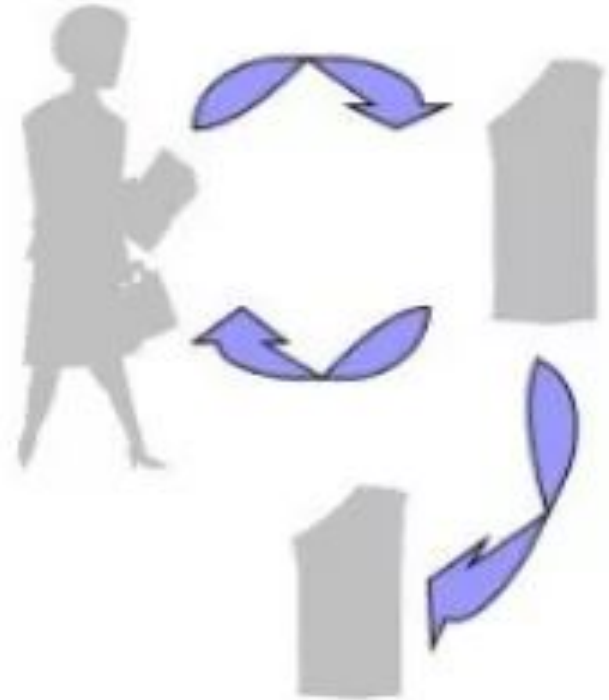
Apart from these, Java is also used to develop software for devices.

It is used not only in computers and mobile devices, but even in electronic devices like televisions, air conditioners, washing machines, and so on.

Online registration forms, banking apps, and shopping via the internet are all possible because of Java.

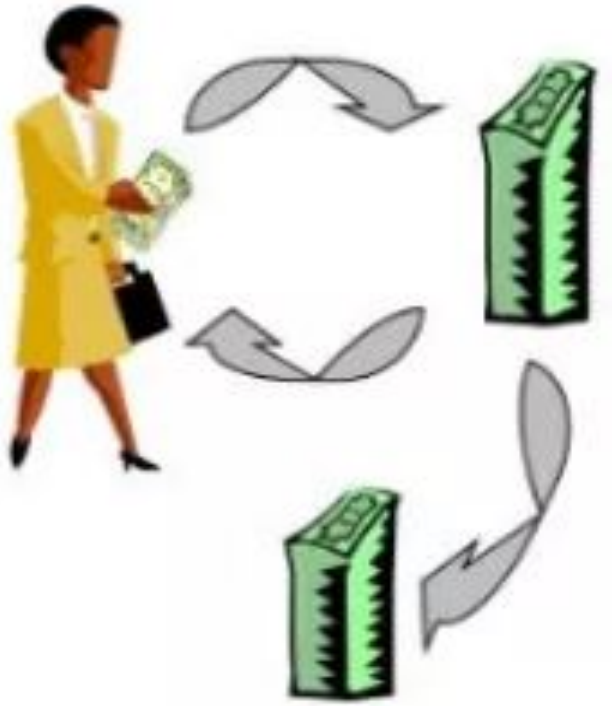
Procedural vs. Object-Oriented

- Procedural



Withdraw, deposit, transfer

- Object Oriented



Customer, money, account

Traditional Procedural Programming VS OOP

Procedural Programming can be defined as a programming model which is derived from structured programming, based upon the concept of calling procedure. Procedures, also known as routines, subroutines or functions, simply consist of a series of computational steps(algorithms) to be carried out. In procedural programming, all functions are treated as separate programs.

COBOL, C, FORTRAN, PASCAL

Object oriented programming can be defined as a programming model which is based upon the concept of objects. Objects contain data in the form of attributes and code in the form of methods. In object oriented programming, computer programs are designed using the concept of objects that interact with real world.

Java, Python, JavaScript, C++,Perl, C#

Objects

- An object represents an entity in the real world such as person, thing or concept etc.
- An object is identified by its name
- An object consists of the following two things

1. **Properties:** are the characteristics / attributes of an object
   - For example: the properties of object **Car** can be
     - Model
     - Color
     - Price
     - Engine Power

Objects

2. Functions: are the actions / task that can be performed by an object

- For example, the object Car can perform the following functions
  - Start
  - Stop
  - Accelerate
  - Reverse
- The set of all functions of an object represents the behavior of the object

Classes

- **A collection of objects with same properties and functions is known as class.**
- A class is used to define the characteristics of the objects.
- It is used as a model for creating different objects of same type.
- For Example
  - A class **Person** can be used to define the characteristics and functions of a person
  - It can be used to create many objects of type Person such as **Ali, Usman, Abdullah** etc.
  - All objects of **Person** class will have same characteristics and functions
  - However, the values of each object can be different and assigned after creating an object

Classes

- Each object of a class is known as instance of its class.
- For example
  - **Ali, Usman and Abdullah** are three instances of a class **Person**

Key Tools for Programming

Editors: Allows user to enter the program. Notepad etc are all editors.

Compilers: Translates the program into target code .

Debuggers: Allows a programmer to run the program to see the execution of the program and correct any errors.

Profilers: Used to evaluate program's performance.

Integrated Development Environment (IDE): Combines editor, compiler, debugger and profiler or a subset into one tool.

- Common Java IDEs are Eclipse, Netbeans, BlueJ, and DrJava.

Characteristics of the Java PL

Simple

Object oriented

Distributed

Multithreaded

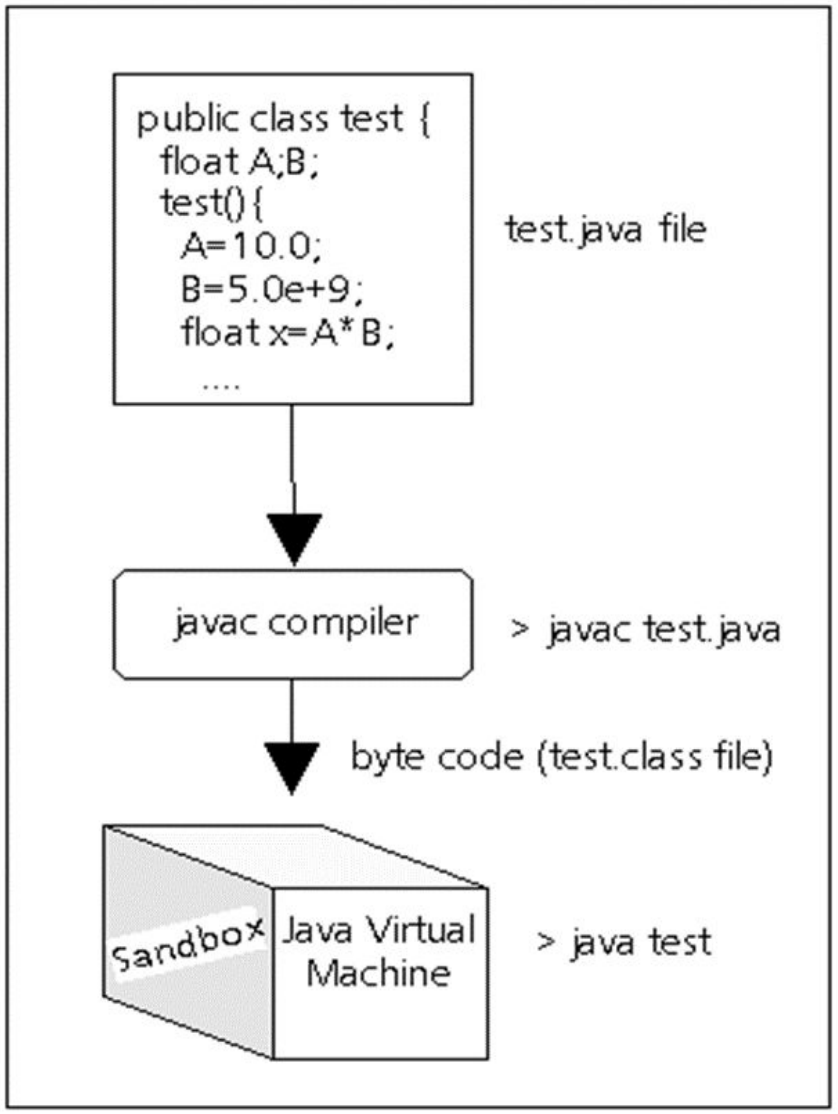
Dynamic

Architecture neutral

Java Procedure

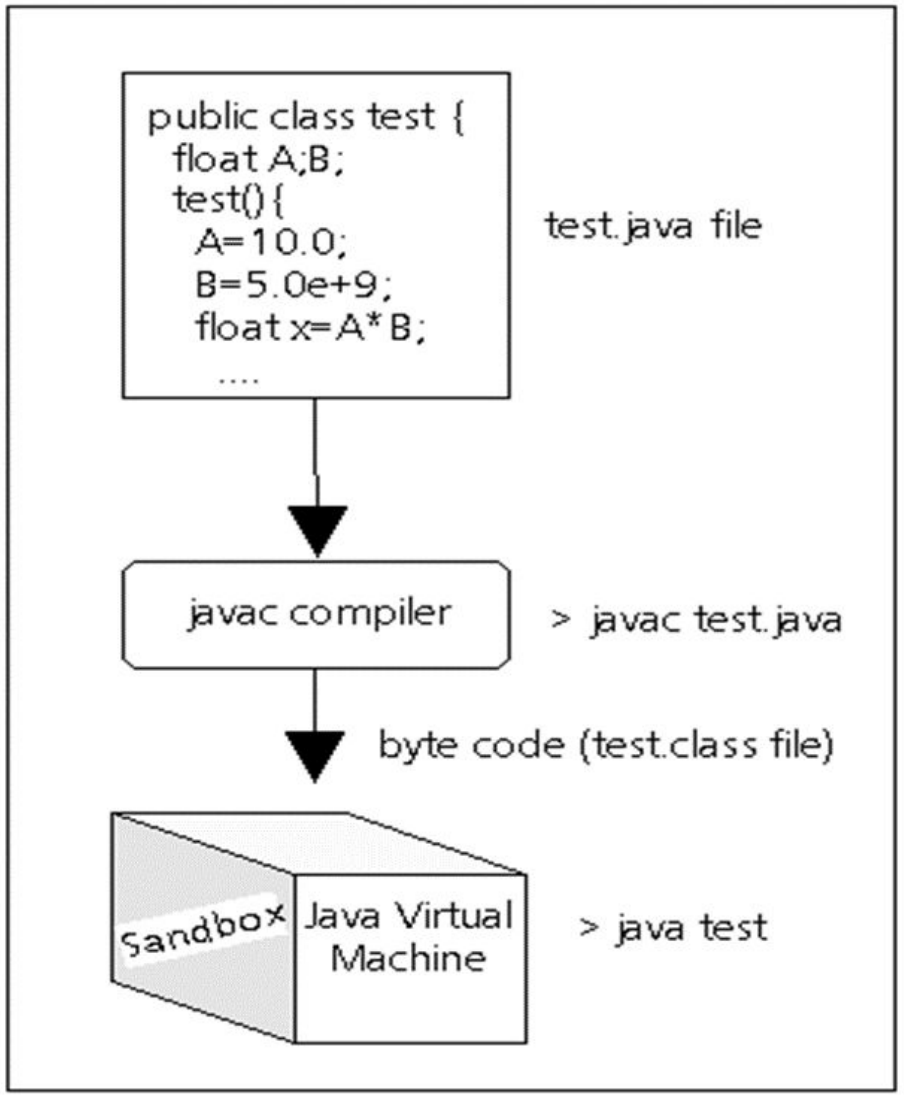
The essential steps to creating and running Java programs go as follows:

- Create a Java source code file
- Compile the source code
- Run the compiled code in a Java Virtual Machine.



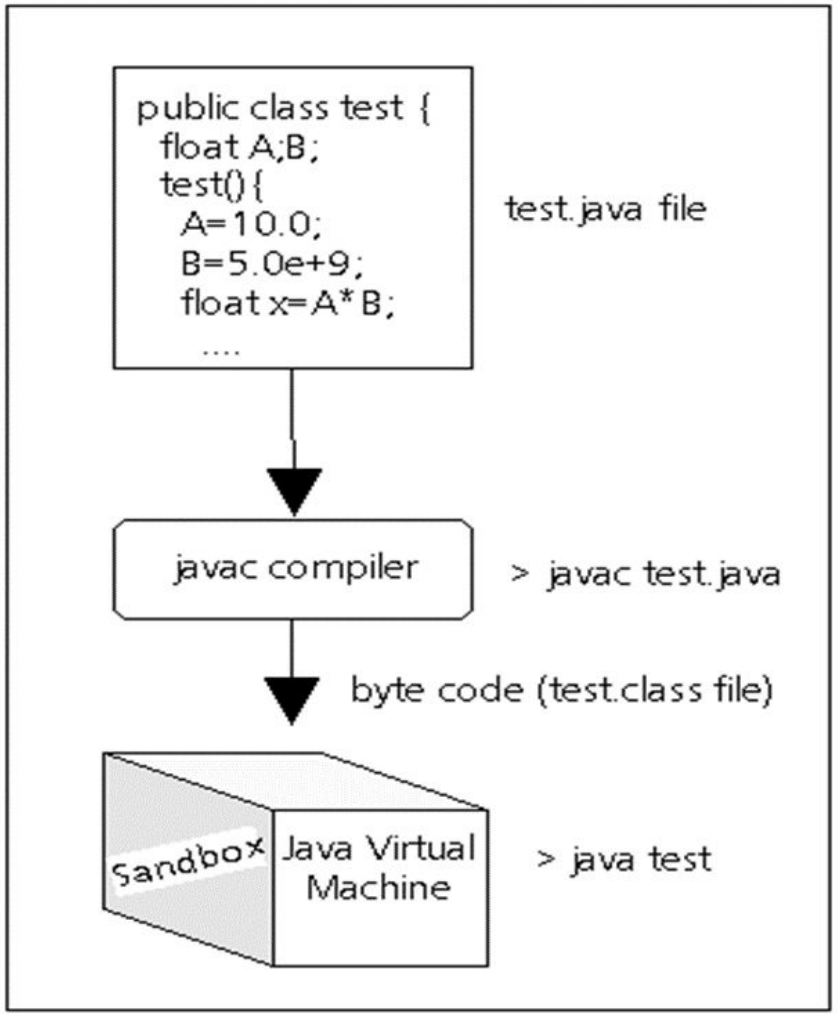
Java Procedure Detail

1. You create the code with a text editor (for example notepad) and save it to a file with the ".java" suffix.
2. All Java source code files must end with this type name.
3. The first part of the name must match the class name in the source code.
4. In the figure the class name is Test so you must therefore save it to the file name Test.java.



Java Procedure Detail

1. With the javac program, you compile this file as follows:
   - C:> javac Test.java
2. This creates a *bytecode* file that ends with the ".class" type appended.
   - Here the output is Test.class.
   - The bytecode consists of the instructions for the *Java Virtual Machine* (JVM or just VM).



Java Procedure Detail

- The JVM is an interpreter program that emulates a processor that executes the bytecode instructions just as if it were a hardware processor executing native machine code instructions.
  - The Java bytecode can then run on any platform in which the JVM is available and the program should perform the same.
  - This *Write Once, Run Anywhere* approach is a key goal of the Java language.

Programming Tools

You can choose from essentially two different *programming environments* for Java:

- **Manual** – with text editor (notepad) create the Java source code files (*.java) and then use the command line tools in the Java *Software Development Kit (SDK)*. The SDK is provided by Sun for several platforms and includes a number of tools, the most important of which are:
  - javac - compiles Java source code files (e.g. Test.java) to bytecode files (e.g. Test.class) *
  - java - runs Java application programs (i.e. implements the JVM for the Java programs.)

Programming Tools

- **Integrated Development Environment** (IDE) - graphical user interface programming environments (often called *GUI Builders*) are elaborate, programs that allow you to interactively build graphical interfaces, edit the code, execute and run the applets and applications all within the IDE system. Example Java IDEs include:
  - **NetBeans**
  - Borland JBuilder
  - **Eclipse**
  - Dr Java
  - **Visual Studio Code**

Examples

Let's create our first Java file, called Main.java, which can be done in any text editor (like Notepad).

The file should contain a "Hello World" message, which is written with the following code:

Main.java

```
public class Main {
  public static void main(String[] args) {
    System.out.println("Hello World");
  }
}
```

Java Syntax

Every line of code that runs in Java must be inside a class. In our example, we named the class **Main**. A class should always start with an uppercase first letter.

Note: Java is case-sensitive: "MyClass" and "myclass" has different meaning.

The name of the java file **must match** the class name. When saving the file, save it using the class name and add ".java" to the end of the filename.

The main Method

The `main()` method is required and you will see it in every Java program:

```
public static void main(String[] args)
```

Any code inside the `main()` method will be executed. Don't worry about the keywords before and after main. You will get to know them bit by bit while reading this tutorial.

For now, just remember that every Java program has a `class` name which must match the filename, and that every program must contain the `main()` method.

Java Output

Inside the `main()` method, we can use the `println()` method to print a line of text to the screen:

```
public static void main(String[] args) {
  System.out.println("Hello World");
}
```

Note: The curly braces `{}` marks the beginning and the end of a block of code.

`System` is a built-in Java class that contains useful members, such as `out`, which is short for "output". The `println()` method, short for "print line", is used to print a value to the screen (or a file).

Cont..

You can add as many `println()` methods as you want. Note that it will add a new line for each method:

Example

```
System.out.println("Hello World!");
System.out.println("I am learning Java.");
System.out.println("It is awesome!");
```

You can also output numbers, and perform mathematical calculations:

Example

```
System.out.println(3 + 3);
```

The Print() Method

There is also a `print()` method, which is similar to `println()`.
The only difference is that it does not insert a new line at the end of the output:

Example

```
System.out.print("Hello World! ");
System.out.print("I will print on the same line.");
```

Java Comments

Comments can be used to explain Java code, and to make it more readable. It can also be used to prevent execution when testing alternative code

Single-line Comments

Single-line comments start with two forward slashes (//).
Any text between // and the end of the line is ignored by Java (will not be executed).
This example uses a single-line comment before a line of code:

Example

```
// This is a comment
System.out.println("Hello World");
```

Java Multi-line Comments

Multi-line comments start with /* and ends with */.
Any text between /* and */ will be ignored by Java.
This example uses a multi-line comment (a comment block) to explain the code:

Example

```
/* The code below will print the words Hello World
to the screen, and it is amazing */
System.out.println("Hello World");
```

Example for Classes and Objects

```java
// Define the Person class
class Person {
    String name;
    int age;

    // Method to display person details
    void display() {
        System.out.println(name + " is " + age + " years old.");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        // Creating and displaying Person objects
        Person p1 = new Person();
        p1.name = "Ali";
        p1.age = 25;

        Person p2 = new Person();
        p2.name = "Usman";
        p2.age = 30;

        p1.display();
        p2.display();
    }
}
```